

DATA STRUCTURES USING PYTHON

Submitted By:

Name: Manvendra Pratap Singh

Year: 2nd

Semester: 4th

Roll No.: 20234011

Assignment

1. Write a program to implement singly linked list as an ADT that supports the following operations:
 - i. Insert an element x at the beginning of the singly linked list

Ans : **class Node:**

```
def __init__(self, data):
```

```
    self.data = data
```

```
    self.next = None
```

class SinglyLinkedList:

```
def __init__(self):
```

```
    self.head = None
```

```
def insert_at_beginning(self, x):
```

```
    new_node = Node(x)
```

```
    new_node.next = self.head
```

```
    self.head = new_node
```

```
def display(self):
```

```
    current = self.head
```

```
    while current:
```

```
        print(current.data, end=" -> ")
```

```
        current = current.next
```

```
    print("None")
```

```
if __name__ == "__main__":
```

```
    sll = SinglyLinkedList()
```

```
sll.insert_at_beginning(10)
sll.insert_at_beginning(20)
sll.insert_at_beginning(30)
sll.display()
```

```
PS C:\Users\lenovo> & C:/p
30 -> 20 -> 10 -> None
```

- ii. Insert an element x at i^{th} position in the singly linked list

Ans:

class Node:

```
def __init__(self, data):
    self.data = data
    self.next = None
```

class SinglyLinkedList:

```
def __init__(self):
    self.head = None

def insert_at_beginning(self, x):
    new_node = Node(x)
    new_node.next = self.head
    self.head = new_node

def insert_at_position(self, x, i):
    new_node = Node(x)
```

```
if i == 0:  
    new_node.next = self.head  
    self.head = new_node  
    return
```

```
current = self.head  
count = 0  
while current is not None and count < i - 1:  
    current = current.next  
    count += 1
```

```
if current is None:  
    print("Position out of bounds!")  
    return  
new_node.next = current.next  
current.next = new_node
```

```
def display(self):  
    current = self.head  
    while current:  
        print(current.data, end=" -> ")  
        current = current.next
```

```

    print("None")

if __name__ == "__main__":
    sll = SinglyLinkedList()

    # Inserting at beginning
    sll.insert_at_beginning(10)
    sll.insert_at_beginning(20)
    sll.insert_at_beginning(30)
    sll.insert_at_position(25, 1) # Insert 25 at position 1
    sll.insert_at_position(5, 4) # Insert 5 at position 4
    sll.insert_at_position(100, 10) # Invalid position
    sll.display()

```

```

Position out of bounds!
30 -> 25 -> 20 -> 10 -> 5 -> None

```

iii. Remove an element from the beginning of the doubly linked list

Ans:

class Node:

```

    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

```

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, x):
        new_node = Node(x)
        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node
        self.head = new_node

    def remove_from_beginning(self):
        if self.head is None:
            print("List is empty! Nothing to remove.")
            return

        if self.head.next is None:
            self.head = None
        else:
            self.head = self.head.next
            self.head.prev = None

    def display(self):
        current = self.head
```

```
while current:
    print(current.data, end=" <-> ")
    current = current.next
print("None")
if __name__ == "__main__":
    dll = DoublyLinkedList()

    dll.insert_at_beginning(10)
    dll.insert_at_beginning(20)
    dll.insert_at_beginning(30)

    print("Original List:")
    dll.display()
    dll.remove_from_beginning()
    print("\nAfter removing one element from beginning:")
    dll.display()
    dll.remove_from_beginning()
    print("\nAfter removing another element from
beginning:")
    dll.display()
    dll.remove_from_beginning()
    print("\nAfter removing last element from beginning:")
```

dll.display()

dll.remove_from_beginning()

```
Original List:
30 <-> 20 <-> 10 <-> None

After removing one element from beginning:
20 <-> 10 <-> None

After removing another element from beginning:
10 <-> None

After removing last element from beginning:
None
List is empty! Nothing to remove.
```

iv. Remove an element from i^{th} position in the singly linked list.

Ans:

class Node:

def __init__(self, data):

self.data = data

self.next = None

class SinglyLinkedList:

def __init__(self):

self.head = None

def insert_at_beginning(self, x):

new_node = Node(x)

```
new_node.next = self.head  
self.head = new_node
```

```
def insert_at_position(self, x, i):  
    new_node = Node(x)  
    if i == 0:  
        new_node.next = self.head  
        self.head = new_node  
    return
```

```
current = self.head  
count = 0
```

```
while current is not None and count < i - 1:  
    current = current.next  
    count += 1
```

```
if current is None:  
    print("Position out of bounds!")  
    return
```

```
new_node.next = current.next
```

```
current.next = new_node
```

```
def remove_at_position(self, i):
```

```
    if self.head is None:
```

```
        print("List is empty! Nothing to remove.")
```

```
        return
```

```
    if i == 0:
```

```
        self.head = self.head.next
```

```
        return
```

```
    current = self.head
```

```
    count = 0
```

```
    while current is not None and count < i - 1:
```

```
        current = current.next
```

```
        count += 1
```

```
    if current is None or current.next is None:
```

```
        print("Position out of bounds!")
```

```
        return
```

```
current.next = current.next.next
```

```
def display(self):
```

```
    current = self.head
```

```
    while current:
```

```
        print(current.data, end=" -> ")
```

```
        current = current.next
```

```
    print("None")
```

```
if __name__ == "__main__":
```

```
    sll = SinglyLinkedList()
```

```
    sll.insert_at_beginning(10)
```

```
    sll.insert_at_beginning(20)
```

```
    sll.insert_at_beginning(30)
```

```
    sll.insert_at_beginning(40)
```

```
    print("Original List:")
```

```
    sll.display()
```

```
    sll.remove_at_position(2)
```

```
    print("\nAfter removing element at position 2:")
```

sll.display()

sll.remove_at_position(5)

```
Original List:  
40 -> 30 -> 20 -> 10 -> None  
  
After removing element at position 2:  
40 -> 30 -> 10 -> None  
Position out of bounds!
```

- v. Search for an element x in the singly linked list and return its pointer

Ans: **class Node:**

def __init__(self, data):

self.data = data

self.next = None

class SinglyLinkedList:

def __init__(self):

self.head = None

def insert_at_beginning(self, x):

new_node = Node(x)

new_node.next = self.head

self.head = new_node

def insert_at_position(self, x, i):

new_node = Node(x)

```
if i == 0:
    new_node.next = self.head
    self.head = new_node
    return

current = self.head
count = 0

while current is not None and count < i - 1:
    current = current.next
    count += 1

if current is None:
    print("Position out of bounds!")
    return

new_node.next = current.next
current.next = new_node

def remove_at_position(self, i):
    if self.head is None:
        print("List is empty! Nothing to remove.")
        return
```

```
if i == 0:  
    self.head = self.head.next  
    return
```

```
current = self.head  
count = 0
```

```
while current is not None and count < i - 1:  
    current = current.next  
    count += 1
```

```
if current is None or current.next is None:  
    print("Position out of bounds!")  
    return
```

```
current.next = current.next.next
```

```
def search(self, x):  
    current = self.head  
    while current is not None:  
        if current.data == x:
```

```

        return current # Return the node (pointer)

    current = current.next

    return None # Not found

def display(self):
    current = self.head
    while current:
        print(current.data, end=" -> ")
        current = current.next
    print("None")

if __name__ == "__main__":
    sll = SinglyLinkedList()
    sll.insert_at_beginning(10)
    sll.insert_at_beginning(20)
    sll.insert_at_beginning(30)
    sll.insert_at_beginning(40)
    print("Original List:")
    sll.display()

    result = sll.search(20)
    if result:
        print(f"\nElement 20 found at node with data:
{result.data}")

```

else:

print("\nElement 20 not found.")

result = sll.search(99)

if result:

**print(f"\nElement 99 found at node with data:
{result.data}")**

else:

print("\nElement 99 not found.")

```
Original List:  
40 -> 30 -> 20 -> 10 -> None  
  
Element 20 found at node with data: 20  
  
Element 99 not found.
```

2. Write a program to implement doubly linked list as an ADT that supports the following operations:
- Insert an element x at the beginning of the doubly linked list
 - Insert an element x at the end of the doubly linked list
 - Remove an element from the beginning of the doubly linked list
 - Remove an element from the end of the doubly linked list

Ans:

class Node:

def __init__(self, data):

```
self.data = data  
self.prev = None  
self.next = None
```

Doubly Linked List class

class DoublyLinkedList:

```
def __init__(self):  
    self.head = None
```

i. Insert at beginning

```
def insert_at_beginning(self, x):  
    new_node = Node(x)  
    new_node.next = self.head  
    if self.head is not None:  
        self.head.prev = new_node  
    self.head = new_node
```

ii. Insert at end

```
def insert_at_end(self, x):  
    new_node = Node(x)  
    if self.head is None:  
        self.head = new_node
```

```
    return
current = self.head
while current.next:
    current = current.next
current.next = new_node
new_node.prev = current
```

iii. Remove from beginning

```
def remove_from_beginning(self):
    if self.head is None:
        print("List is empty! Nothing to remove.")
        return
    if self.head.next is None:
        self.head = None
    else:
        self.head = self.head.next
        self.head.prev = None
```

iv. Remove from end

```
def remove_from_end(self):
    if self.head is None:
        print("List is empty! Nothing to remove.")
```

```

        return
    if self.head.next is None:
        self.head = None
        return
    current = self.head
    while current.next:
        current = current.next
    current.prev.next = None

# Display list
def display(self):
    current = self.head
    while current:
        print(current.data, end=" <-> ")
        current = current.next
    print("None")

# Test the doubly linked list
if __name__ == "__main__":
    dll = DoublyLinkedList()

# Insertions

```

```
dll.insert_at_beginning(10)
```

```
dll.insert_at_beginning(20)
```

```
dll.insert_at_end(5)
```

```
dll.insert_at_end(1)
```

```
print("List after insertions:")
```

```
dll.display()
```

```
# Removals
```

```
dll.remove_from_beginning()
```

```
print("\nList after removing from beginning:")
```

```
dll.display()
```

```
dll.remove_from_end()
```

```
print("\nList after removing from end:")
```

```
dll.display()
```

```
dll.remove_from_beginning()
```

```
dll.remove_from_end()
```

```
print("\nList after removing all elements:")
```

```
dll.display()
```

dll.remove_from_beginning()

```
List after insertions:  
20 <-> 10 <-> 5 <-> 1 <-> None  
  
List after removing from beginning:  
10 <-> 5 <-> 1 <-> None  
  
List after removing from end:  
10 <-> 5 <-> None  
  
List after removing all elements:  
None  
List is empty! Nothing to remove.
```

3. Write a program to implement circular linked list as an ADT which supports the following operations:

- i. Insert an element x in the list
- ii. Remove an element from the list
- iii. Search for an element x in the list and return its pointer

Ans:

class Node:

def __init__(self, data):

self.data = data

self.next = None

class CircularLinkedList:

def __init__(self):

self.head = None

i. Insert an element x in the list (at end)

def insert(self, x):

new_node = Node(x)

if self.head is None:

self.head = new_node

new_node.next = self.head # Point to itself

else:

current = self.head

while current.next != self.head:

current = current.next

current.next = new_node

new_node.next = self.head

ii. Remove an element x from the list

def remove(self, x):

if self.head is None:

print("List is empty! Cannot remove.")

return

current = self.head

prev = None

```
while True:
    if current.data == x:
        if prev is None: # removing head
            # Single node
            if current.next == self.head:
                self.head = None
            else:
                # Find last node to update its next
                tail = self.head
                while tail.next != self.head:
                    tail = tail.next
                self.head = self.head.next
                tail.next = self.head
        else:
            prev.next = current.next
        return
    prev = current
    current = current.next
    if current == self.head:
        break

print(f"Element {x} not found.")
```

iii. Search for an element x and return its pointer

def search(self, x):

if self.head is None:

return None

current = self.head

while True:

if current.data == x:

return current

current = current.next

if current == self.head:

break

return None

Display the list

def display(self):

if self.head is None:

print("List is empty.")

return

current = self.head

```
while True:  
    print(current.data, end=" -> ")  
    current = current.next  
    if current == self.head:  
        break  
print("(back to head)")
```

Usage

```
if __name__ == "__main__":  
    cll = CircularLinkedList()
```

Insert elements

```
cll.insert(10)  
cll.insert(20)  
cll.insert(30)  
cll.insert(40)
```

```
print("Circular Linked List:")  
cll.display()
```

Search for an element

```
result = cll.search(30)
```

```
if result:
    print(f"\nElement 30 found at node with data:
{result.data}")
else:
    print("\nElement 30 not found.")
```

Remove an element

```
cll.remove(20)
print("\nAfter removing element 20:")
cll.display()
```

```
cll.remove(10)
print("\nAfter removing head (10):")
cll.display()
```

```
cll.remove(99)
print("\nTrying to remove non-existing element 99:")
cll.display()
```

```
Circular Linked List:
10 -> 20 -> 30 -> 40 -> (back to head)

Element 30 found at node with data: 30

After removing element 20:
10 -> 30 -> 40 -> (back to head)

After removing head (10):
30 -> 40 -> (back to head)
Element 99 not found.

Trying to remove non-existing element 99:
30 -> 40 -> (back to head)
```

4. Implement Stack as an ADT and use it to evaluate a prefix/postfix expression.

Ans: **class Stack:**

def __init__(self):

self.items = []

def push(self, x):

self.items.append(x)

def pop(self):

if not self.is_empty():

return self.items.pop()

raise IndexError("pop from empty stack")

```
def peek(self):  
    if not self.is_empty():  
        return self.items[-1]  
    return None
```

```
def is_empty(self):  
    return len(self.items) == 0
```

```
def evaluate_postfix(expression):  
    stack = Stack()  
    tokens = expression.split()  
  
    for token in tokens:  
        if token.isdigit():  
            stack.push(int(token))  
        else:  
            right = stack.pop()  
            left = stack.pop()  
            result = apply_operator(left, right, token)  
            stack.push(result)
```

```
return stack.pop()
```

```
def evaluate_prefix(expression):
```

```
    stack = Stack()
```

```
    tokens = expression.split()[::-1] # Reverse the  
    expression
```

```
    for token in tokens:
```

```
        if token.isdigit():
```

```
            stack.push(int(token))
```

```
        else:
```

```
            left = stack.pop()
```

```
            right = stack.pop()
```

```
            result = apply_operator(left, right, token)
```

```
            stack.push(result)
```

```
    return stack.pop()
```

```
def apply_operator(a, b, operator):
```

```
    if operator == '+': return a + b
```

```
    if operator == '-': return a - b
```

```
    if operator == '*': return a * b
```

```
if operator == '/': return a // b # Integer division
raise ValueError(f"Unknown operator: {operator}")
```

```
if __name__ == "__main__":
    postfix_expr = "5 6 2 + * 12 4 / -"
    prefix_expr = "- * 5 + 6 2 / 12 4"

    print("Postfix Evaluation:")
    print(f"Expression: {postfix_expr}")
    print("Result:", evaluate_postfix(postfix_expr))

    print("\nPrefix Evaluation:")
    print(f"Expression: {prefix_expr}")
    print("Result:", evaluate_prefix(prefix_expr))
```

```
Postfix Evaluation:
Expression: 5 6 2 + * 12 4 / -
Result: 37

Prefix Evaluation:
Expression: - * 5 + 6 2 / 12 4
Result: 37
```

5. Implement Queue as an ADT.

Ans: **class Queue:**

```
    def __init__(self):
```

```
self.items = []

# Enqueue operation (add to rear)
def enqueue(self, x):
    self.items.append(x)

# Dequeue operation (remove from front)
def dequeue(self):
    if not self.is_empty():
        return self.items.pop(0)
    raise IndexError("dequeue from empty queue")

# Peek front element
def front(self):
    if not self.is_empty():
        return self.items[0]
    return None

# Check if queue is empty
def is_empty(self):
    return len(self.items) == 0
```

Get size of queue

def size(self):

return len(self.items)

Display the queue

def display(self):

print("Front ->", end=" ")

for item in self.items:

print(item, end=" ")

print("<- Rear")

if __name__ == "__main__":

q = Queue()

q.enqueue(10)

q.enqueue(20)

q.enqueue(30)

print("Initial queue:")

q.display()

print("Front element:", q.front())

```
print("Dequeue:", q.dequeue())
```

```
print("Queue after dequeue:")
```

```
q.display()
```

```
print("Queue size:", q.size())
```

```
print("Is queue empty?", q.is_empty())
```

```
Initial queue:  
Front -> 10 20 30 <- Rear  
Front element: 10  
Dequeue: 10  
Queue after dequeue:  
Front -> 20 30 <- Rear  
Queue size: 2  
Is queue empty? False
```

6. Write a program to implement Binary Search Tree as an ADT which supports the following operations:
- Insert an element x
 - Delete an element x
 - Search for an element x in the BST
 - Display the elements of the BST in preorder, inorder and postorder traversal

Ans: **class Node:**

```
def __init__(self, key):
```

```
    self.key = key
```

```
    self.left = None
```

```
self.right = None
```

```
# BST class
```

```
class BinarySearchTree:
```

```
    def __init__(self):
```

```
        self.root = None
```

```
# i. Insert an element
```

```
def insert(self, key):
```

```
    self.root = self._insert_recursive(self.root, key)
```

```
def _insert_recursive(self, node, key):
```

```
    if node is None:
```

```
        return Node(key)
```

```
    if key < node.key:
```

```
        node.left = self._insert_recursive(node.left, key)
```

```
    elif key > node.key:
```

```
        node.right = self._insert_recursive(node.right, key)
```

```
    return node
```

```
# ii. Delete an element
```

```
def delete(self, key):
```

```
self.root = self._delete_recursive(self.root, key)
```

```
def _delete_recursive(self, node, key):
```

```
    if node is None:
```

```
        return None
```

```
    if key < node.key:
```

```
        node.left = self._delete_recursive(node.left, key)
```

```
    elif key > node.key:
```

```
        node.right = self._delete_recursive(node.right, key)
```

```
    else:
```

```
        # Case 1: no child
```

```
        if node.left is None and node.right is None:
```

```
            return None
```

```
        # Case 2: one child
```

```
        if node.left is None:
```

```
            return node.right
```

```
        if node.right is None:
```

```
            return node.left
```

```
        # Case 3: two children
```

```
        min_larger_node = self._min_value_node(node.right)
```

```
        node.key = min_larger_node.key
```

```
        node.right = self._delete_recursive(node.right,  
min_larger_node.key)  
    return node
```

```
def _min_value_node(self, node):  
    current = node  
    while current.left:  
        current = current.left  
    return current
```

iii. Search for an element

```
def search(self, key):  
    return self._search_recursive(self.root, key)
```

```
def _search_recursive(self, node, key):  
    if node is None or node.key == key:  
        return node  
    if key < node.key:  
        return self._search_recursive(node.left, key)  
    else:  
        return self._search_recursive(node.right, key)
```

iv. Traversals

```
def inorder(self):
```

```
    print("Inorder:", end=" ")
```

```
    self._inorder(self.root)
```

```
    print()
```

```
def _inorder(self, node):
```

```
    if node:
```

```
        self._inorder(node.left)
```

```
        print(node.key, end=" ")
```

```
        self._inorder(node.right)
```

```
def preorder(self):
```

```
    print("Preorder:", end=" ")
```

```
    self._preorder(self.root)
```

```
    print()
```

```
def _preorder(self, node):
```

```
    if node:
```

```
        print(node.key, end=" ")
```

```
        self._preorder(node.left)
```

```
        self._preorder(node.right)
```

```
def postorder(self):  
    print("Postorder:", end=" ")  
    self._postorder(self.root)  
    print()
```

```
def _postorder(self, node):  
    if node:  
        self._postorder(node.left)  
        self._postorder(node.right)  
        print(node.key, end=" ")
```

```
if __name__ == "__main__":
```

```
    bst = BinarySearchTree()
```

```
    # Insert elements
```

```
    for key in [50, 30, 70, 20, 40, 60, 80]:
```

```
        bst.insert(key)
```

```
    # Traversals
```

```
    bst.inorder() # 20 30 40 50 60 70 80
```

```
    bst.preorder() # 50 30 20 40 70 60 80
```

```
bst.postorder() # 20 40 30 60 80 70 50
```

```
result = bst.search(60)
```

```
print("\nSearch 60:", "Found" if result else "Not found")
```

```
result = bst.search(99)
```

```
print("Search 99:", "Found" if result else "Not found")
```

```
bst.delete(30)
```

```
print("\nAfter deleting 30:")
```

```
bst.inorder()
```

```
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
```

```
Search 60: Found
Search 99: Not found
```

```
After deleting 30:
Inorder: 20 40 50 60 70 80
```

7. Write a program to implement insert and search operation in AVL trees.

Ans: **class AVLNode:**

```
def __init__(self, key):
```

```
self.key = key
self.left = None
self.right = None
self.height = 1 # New node is initially added at leaf
```

AVL Tree class

class AVLTree:

```
def __init__(self):
```

```
    self.root = None
```

Get the height of a node

```
def _height(self, node):
```

```
    return node.height if node else 0
```

Get balance factor of a node

```
def _get_balance(self, node):
```

```
    return self._height(node.left) - self._height(node.right)
if node else 0
```

Right rotate

```
def _right_rotate(self, y):
```

```
    x = y.left
```

T2 = x.right

x.right = y

y.left = T2

y.height = max(self._height(y.left), self._height(y.right))
+ 1

x.height = max(self._height(x.left), self._height(x.right))
+ 1

return x

Left rotate

def _left_rotate(self, x):

y = x.right

T2 = y.left

y.left = x

x.right = T2

x.height = max(self._height(x.left), self._height(x.right))
+ 1

```
y.height = max(self._height(y.left), self._height(y.right))  
+ 1
```

```
return y
```

```
# Insert a node
```

```
def insert(self, key):
```

```
    self.root = self._insert(self.root, key)
```

```
def _insert(self, node, key):
```

```
    # 1. Perform normal BST insertion
```

```
    if not node:
```

```
        return AVLNode(key)
```

```
    elif key < node.key:
```

```
        node.left = self._insert(node.left, key)
```

```
    elif key > node.key:
```

```
        node.right = self._insert(node.right, key)
```

```
    else:
```

```
        return node # Duplicate keys not allowed
```

```
# 2. Update the height of the ancestor node
```

```
node.height = 1 + max(self._height(node.left),
self._height(node.right))
```

3. Get the balance factor

```
balance = self._get_balance(node)
```

4. Balance the tree

Case 1: Left Left

```
if balance > 1 and key < node.left.key:
```

```
    return self._right_rotate(node)
```

Case 2: Right Right

```
if balance < -1 and key > node.right.key:
```

```
    return self._left_rotate(node)
```

Case 3: Left Right

```
if balance > 1 and key > node.left.key:
```

```
    node.left = self._left_rotate(node.left)
```

```
    return self._right_rotate(node)
```

Case 4: Right Left

```
if balance < -1 and key < node.right.key:
```

```
node.right = self._right_rotate(node.right)
return self._left_rotate(node)
```

```
return node
```

Search for a node

```
def search(self, key):
    return self._search(self.root, key)
```

```
def _search(self, node, key):
    if node is None or node.key == key:
        return node
    if key < node.key:
        return self._search(node.left, key)
    else:
        return self._search(node.right, key)
```

Inorder traversal (for display)

```
def inorder(self):
    print("Inorder Traversal:", end=" ")
    self._inorder(self.root)
    print()
```

```
def _inorder(self, node):  
    if node:  
        self._inorder(node.left)  
        print(node.key, end=" ")  
        self._inorder(node.right)
```

```
if __name__ == "__main__":  
    avl = AVLTree()  
    keys = [10, 20, 30, 40, 50, 25]
```

```
    for key in keys:  
        avl.insert(key)
```

```
    avl.inorder() # Output should be sorted
```

```
    x = 25
```

```
    found = avl.search(x)
```

```
    if found:
```

```
        print(f"Element {x} found in AVL tree.")
```

```
    else:
```

```
        print(f"Element {x} not found.")
```

```
Inorder Traversal: 10 20 25 30 40 50  
Element 25 found in AVL tree.
```